

P1225R0

Feedback on 2D Graphics

Published Proposal, 2018-10-02

This version:

<http://wg21.link/P1225R0>

Author:

[JF Bastien](#) (Apple)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/jfbastien/papers/blob/master/source/P1225R0.bs

Table of Contents

- 1 Abstract
- 2 Design
- 3 Science and teaching
- 4 Abstraction Level
- 5 Missing Details
- 6 C++ Aesthetics
- 7 Conclusion

References

Informative References

§ 1. Abstract

I've gathered input from a variety of folks involved in graphics at Apple, and here is our joint, considered, position regarding the 2D Graphics proposal.

We're worried that the 2D Graphics proposal in [\[P0267R8\]](#) might be detrimental to developers, students, and users of devices which contain C++ code. Graphics are important to the Apple ecosystem, and we can see them as an important part of C++. However, we don't think P0267R8 meets the quality bar for acceptance into C++. We want to see the reference implementation prove orthogonality, extensibility, and performance across a handful of platforms.

§ 2. Design

Were we to design a 2D Graphics API, we'd do the following:

1. Multiple output devices: Memory buffer, Window, SVG, PDF, etc.
 1. Memory buffer must be directly usable by graphics API
 2. Support types such as `fp16` [\[P0303R0\]](#)
 3. Alpha channel support
2. Anti-aliasing should come for free where supported
3. Text
4. Consistent, DPI-independent, output
5. Hardware support where available
6. Reasonable performance
7. Reasonable power consumption
8. Color spaces and gamma support
9. Possibility to build an interactive model with animation on top of the API

From the current proposal we like:

1. 2D Matrix is 3×3 , so homogeneous, presented as 2×3 in the API
2. Decouples display points from actual points
3. Vector graphics
4. Compositing properly handled

§ 3. Science and teaching

We've heard the following reasons for including 2D Graphics in C++:

1. Teaching
2. Scientific plot generation

We think putting pixels on the screen is great, but we want to do so responsibly.

Both for science and teaching, we appreciate what's available through solutions such as [Matlab](#) / [matplotlib](#) / [R](#) / [D3.js](#). These solutions are powerful and match the performance of the language they complement. For C++ we'd expect a solution which is able to deliver performance which at least approaches that of modern graphics frameworks, and surpassing those of Matlab / Python / R / JavaScript.

As a teaching tool, the current proposal teaches fairly low-level capabilities (i.e. complex things are hard to create) and is missing critical functionality. We fear it will hinder students by teaching them to start everything from scratch, and by not teaching them a few key details.

As a plotting tool it's clearly falling short because it can't label any axis (c.f. [Tufte](#)). Even if text were supported, the sample libraries for Matlab, Python, R, and JavaScript are much easier to draw plots with. The 2D Graphics proposal is neither capable nor convenient in that regard.

As a broad generalization, students currently learn data visualization (beyond what Excel + CSV files can do) in Matlab or Python if they do science, in R if they do math, and in JavaScript if they do anything else. We urge the Committee members at least try some of these, for example [scatterplot](#), [histogram](#), [wordtree](#). These aren't teaching toys and are used, for example, by the [New York Times](#). There's value in teaching students to pull themselves up from the language's bootstraps, we therefore think the type of API in the current 2D Graphics library is useful. However, we want to know—i.e. we want to see it prototyped—that higher-level capabilities are also something that can be implemented. We think higher-level capabilities are more useful for teaching, yet we understand that C++ might want to offer lower-level primitives first.

§ 4. Abstraction Level

When we say the current proposal is too low-level, here are things we'd like to see at least prototyped to know that the proposal can grow into a powerful high-level library:

- Obtain a window object
- Load / transform / draw asset files

- Complex raster image support (including swizzled surfaces, compression, 2D form clipping, used as texture fill)
- New user-implemented rasterization primitives (such as ellipses or NURBS curve)
- Stacking geometric transforms before drawing (can this be done already?)
- Scissoring / clipping
- Handle user input
- Text support (glyph rasterization (e.g. FreeType), text Shaping (e.g. HarfBuzz), string Rendering (e.g. Pango)), or something platform specific (e.g. CoreText on Apple Platforms)
- Complex line drawing (e.g. dashed lines, along a path)
- Can all of the offered primitives be implemented directly on hardware using shaders?

In other words, we understand that a proposal might want to start small and grow more features over time. We want to know that this growth is possible, and that features can be composed into higher-level primitives.

§ 5. Missing Details

When we say the current proposal has key details we find missing, here are what we want to see in an initial version:

- It's unclear that buffering is implementable, and that's critical to a high-performance implementation. We'd like to see it implemented. We want to see a deferred mode implementation, not just immediate mode.
- Support modern color spaces and gamma.
- DPI independence is needed.
- Display points seem to address individual pixels in the image. We'd like to be able to address at finer granularity (MSAA samples, typographer points, pica).
- We're not convinced that animation can be supported efficiently (i.e. update a single matrix in the stack of transforms).
- The current proposal doesn't specify which image format can be loaded, yet the reference implementation has PNG, JPEG, TIFF. This lack of specification makes portability difficult.
- We want to see an implementation generate PDF, SVG, raster output, as well as output in an OS window. This should be doable portably with zero code change.

§ 6. C++ Aesthetics

Aesthetically, this lacks the feel of a C++ standard library. In particular:

- The dual error handling mechanism, while reminiscent of filesystem, is quaint in the STL.
- Most APIs seem to be function-oriented and have a C API feel to them.
- We're surprised that we don't have iterators / ranges for e.g. a path. We'd expect STL algorithms to work on such primitives.
- We'd like to see linear algebra, trigonometry, and matrix math standardized separately.

§ 7. Conclusion

We want to offer developers a graphics solution which allows usage of the full capabilities of the hardware we ship, without wasting battery life. Were we to ship the 2D Graphics proposal, we'd be putting our and C++'s good name on an API. We want to be sure it doesn't provide a disservice to developers and users.

We're surprised and worried that the reference implementation on Mac requires X11 and MacPorts. We want to see an implementation that re-uses platform primitives on more than Linux. What was the experience with [CoreGraphics](#)?

The windows + SVG proposal in [\[P1062R0\]](#) isn't terrible. Obtaining a window seems like a simple step forward. SVG has some upsides and a few downsides, but overall we're positive on them. We like that the proposal leans on existing standards.

Web view from [\[P1108R0\]](#) is trivial to support if specified well, but we don't think it does what graphics enthusiasts want to do. It might be an interesting proposal, but we think it stands separately from 2D Graphics.

§ References

§ Informative References

[P0267R8]

Michael B. McLaughlin, Herb Sutter, Jason Zink, Guy Davidson, Michael Kazakov. [A Proposal to Add 2D Graphics Rendering and Display to C++](#). 26 June 2018. URL: <https://wg21.link/p0267r8>

[P0303R0]

Boris Fomitchev, Sergei Nikolaev, Olivier Giroux, Lawrence Cowl. [Extensions to C++ for Short Float Type](#). 15 October 2017. URL: <https://wg21.link/p0303r0>

[P1062R0]

Bryce Adelstein Leibach, Olivier Giroux, Zach Laine, Corentin Jabot, Vittorio Romeo. [Diet Graphics](#). 7 May 2018. URL: <https://wg21.link/p1062r0>

[P1108R0]

Hal Finkel. [web_view](#). 22 June 2018. URL: <https://wg21.link/p1108r0>